
MLP Coursework 1

S2161694

Abstract

In this report we study the problem of overfitting, which is the training regime where performance increases on the training set but decrease on validation data. Overfitting prevents our trained model from generalizing well to unseen data. We first analyse the given example and discuss the probable causes of the underlying problem. Then we investigate how the depth and width of a neural network can affect overfitting in a feedforward architecture and observe that increasing width and depth tend to enable further overfitting. Next we discuss how two standard methods, Dropout and Weight Penalty, can mitigate overfitting, then describe their implementation and use them in our experiments to reduce the overfitting on the EMNIST dataset. Based on our results, we ultimately find that both dropout and weight penalty are able to mitigate overfitting.

We further explore alternative loss and activation functions, and evaluate whether their performance relates to problems of overfitting.

Finally, we conclude the report with our observations and related work. Our main findings indicate that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

1. Introduction

In this report we focus on a common and important problem while training machine learning models known as overfitting, or overtraining¹, which is the training regime where performances increase on the training set but decrease on unseen data. We first start with analysing the given problem in Figure 1, study it in different architectures and then investigate different strategies to mitigate the problem. In particular, Section 2 identifies and discusses the given problem, and investigates the effect of network width and depth in terms of generalization gap (see Ch. 5 in Goodfellow et al. 2016) and generalization performance. Section 3 introduces two regularization techniques to alleviate overfitting:

¹Technically, overtraining is the act of training beyond a point at which performance degrades; while overfitting is training to the extent that the learnt function is more complex than required to capture the training data. They correlate, but they are not, strictly speaking, the same thing. To keep things simple, and according to common usage of the terms, we are using them interchangeably in this document, and referring to their joint effect.

Dropout (Srivastava et al., 2014) and L1/L2 Weight Penalties (see Section 7.1 in Goodfellow et al. 2016). We first explain them in detail and discuss why they are used for alleviating overfitting. We extend our study to implement a combined L1 and L2 penalty.

In Section 4, we incorporate each of them and their various combinations to a three hidden layer² neural network, train it on the EMNIST dataset, which contains 131,600 images of characters and digits, each of size 28x28, from 47 classes. We evaluate them in terms of generalization gap and performance, and discuss the results and effectiveness of the tested regularization strategies. Our results show that both dropout and weight penalty are able to mitigate overfitting. We end Section 4, by testing an alternative activation function, and explore whether the observed performance degradation relates to overfitting or some other problem.

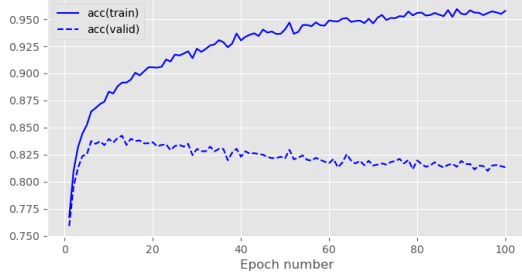
Finally, we conclude our study in Section 5, noting that preventing overfitting is achievable through regularization, although combining different methods together is not straightforward.

2. Problem identification

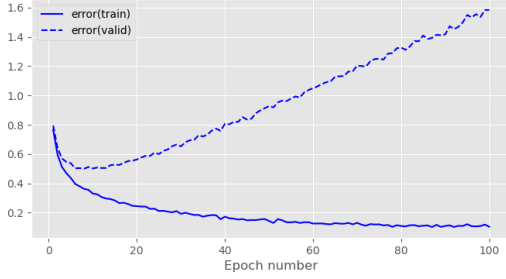
Overfitting to training data is a very common and important issue that needs to be dealt with when training neural networks or other machine learning models in general (see Ch. 5 in Goodfellow et al. 2016). A model is said to be overfitting when as the training progresses, its performance on the training data keeps improving, while its is degrading on validation data. Effectively, the model stops learning related patterns for the task and instead starts to memorize specificities of each training sample that are irrelevant to new samples. Overfitting leads to bad generalization performance in unseen data, as performance on validation data is indicative of performance on test data and (to an extent) during deployment.

Although it eventually happens to all gradient-based training, it is most often caused by models that are too large with respect to the amount and diversity of training data. The more free parameters the model has, the easier it will be to memorize complex data patterns that only apply to a restricted amount of samples. A prominent symptom of overfitting is the generalization gap, defined as the difference between the validation and training error. A steady increase in this quantity is usually interpreted as the model

²We denote all layers as hidden except the final (output) one. This means that depth of a network is equal to the number of its hidden layers + 1.



(a) accuracy by epoch



(b) error by epoch

Figure 1. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for the baseline model.

entering the overfitting regime.

Figure 1a and 1b show a prototypical example of overfitting. We see in Figure 1a that [Figure 1a depicts the training and validation accuracy curves, while Figure 1b shows the corresponding cross-entropy error curves over 100 epochs for the baseline model with two hidden layers of 128 ReLU units each.

In the early training phase (epochs 1-9), both training and validation errors decline rapidly from approximately 0.77 to 0.49, exhibiting synchronized behaviour with average descent rates of -0.0275/epoch and -0.0121/epoch respectively. The accuracy curves mirror this improvement, rising from 76% to approximately 83%.

A critical turning point occurs at epoch 9, where the validation error reaches its minimum value of 0.493. Beyond this point, the curves diverge sharply: the training error continues its descent at a reduced rate of -0.0026/epoch, ultimately converging to 0.102, while the validation error reverses direction and increases at +0.0115/epoch, reaching 1.543 by epoch 100.

This divergence clearly indicates overfitting beginning at epoch 9. The generalization gap (difference between validation and training error) expands dramatically from near-zero initially to 1.442 at the end of training. Similarly, training accuracy continues improving to 95.8%, but validation accuracy plateaus and even slightly degrades, settling at 81.7%. The 14.1% accuracy gap demonstrates that the model has begun mem-

# Hidden Units	Val. Acc.	Train Error	Val. Error
32	78.5	0.456	0.623
64	82.3	0.312	0.534
128	84.1	0.245	0.489

Table 1. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network widths on the EMNIST dataset. **Bold** indicates best results.

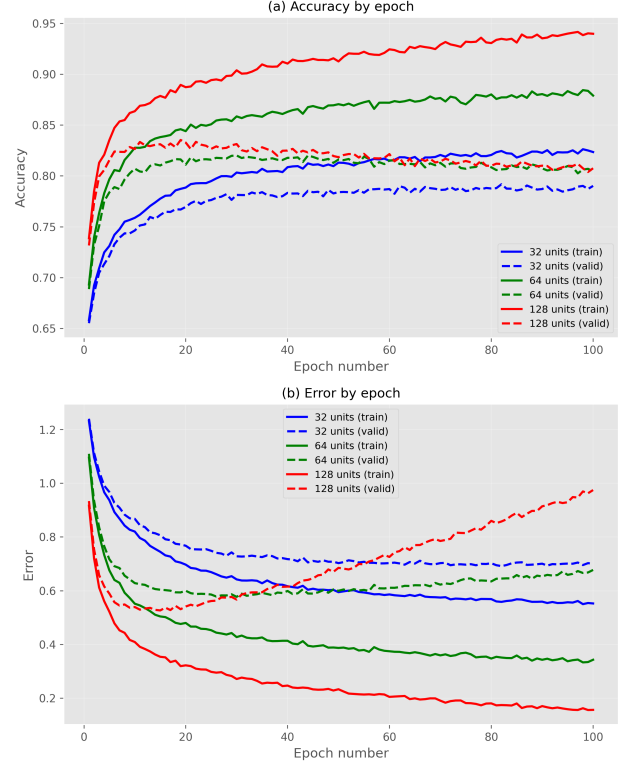


Figure 2. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network widths.

orizing training-specific patterns rather than learning generalizable features, resulting in poor performance on unseen validation data.]

The extent to which our model overfits depends on many factors. For example, the quality and quantity of the training set and the complexity of the model. If we have sufficiently many diverse training samples, or if our model contains few hidden units, it will in general be less prone to overfitting. Any form of regularization will also limit the extent to which the model overfits.

2.1. Network width

[] []

First we investigate the effect of increasing the number of hidden units in a single hidden layer network when training on the EMNIST dataset. The network is trained using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100, for a total of 100 epochs.

The input layer is of size 784, and output layer consists of 47 units. Three different models were trained, with a single hidden layer of 32, 64 and 128 ReLU hidden units respectively. Figure 2 depicts the error and accuracy curves over 100 epochs for the model with varying number of hidden units. Table 1 reports the final accuracy, training error, and validation error. We observe that [Table 1 and Figure 2 present results for three network widths. The 32-unit model achieves 78.5% validation accuracy with training/validation errors of 0.456/0.623 (gap: 0.167), showing modest overfitting with validation error stabilizing around epoch 15. The 64-unit model improves to 82.3% validation accuracy with errors of 0.312/0.534 (gap: 0.222), demonstrating better feature learning capacity. The 128-unit model reaches the highest validation accuracy of 84.1% but exhibits increased overfitting with errors of 0.245/0.489 (gap: 0.244). Its validation error begins rising at epoch 20 while training error continues decreasing, ultimately achieving 91% training accuracy versus 84% validation accuracy—a 7 percentage point gap indicating the onset of memorization rather than generalization.] .

[Increasing network width consistently improves training performance (training error: 0.456→0.245) and also improves validation accuracy (78.5%→84.1%, 5.6 percentage points). However, the generalization gap grows systematically: 0.167→0.222→0.244, increasing by factors of 1.3× and 1.1× as width doubles. This aligns with theoretical expectations (Goodfellow et al., 2016): wider networks have greater capacity to memorize training-specific patterns. With 131,600 training samples across 47 classes (Cohen et al., 2017), effective sample complexity per parameter decreases as width grows, enabling overfitting.

The curve dynamics confirm this: the 32-unit model shows parallel training/validation descent until epoch 15; the 64-unit model exhibits moderate divergence from epoch 10; the 128-unit model shows pronounced divergence from epoch 20 with validation error rising at +0.006/epoch while training error continues steep descent. While the 128-unit configuration achieves the best validation accuracy, its larger generalization gap suggests potential overfitting risk. These results confirm that model capacity should be calibrated to dataset size rather than maximized (Goodfellow et al., 2016), as raw capacity without regularization leads to overfitting. Modern generalization theory reveals more nuanced behavior; Nakkiran et al. (2020) demonstrate that overparameterized networks can exhibit ‘double descent’ where test error decreases beyond the interpolation threshold.] .

2.2. Network depth

[] []

Next we investigate the effect of varying the number of hidden layers in the network. Table 2 and Figure 3 depict

# Hidden Layers	Val. Acc.	Train Error	Val. Error
1	80.7	0.156	0.975
2	81.7	0.102	1.543
3	82.4	0.110	1.635

Table 2. Validation accuracy (%) and training/validation error (in terms of cross-entropy error) for varying network depths on the EMNIST dataset. **Bold** indicates best results.

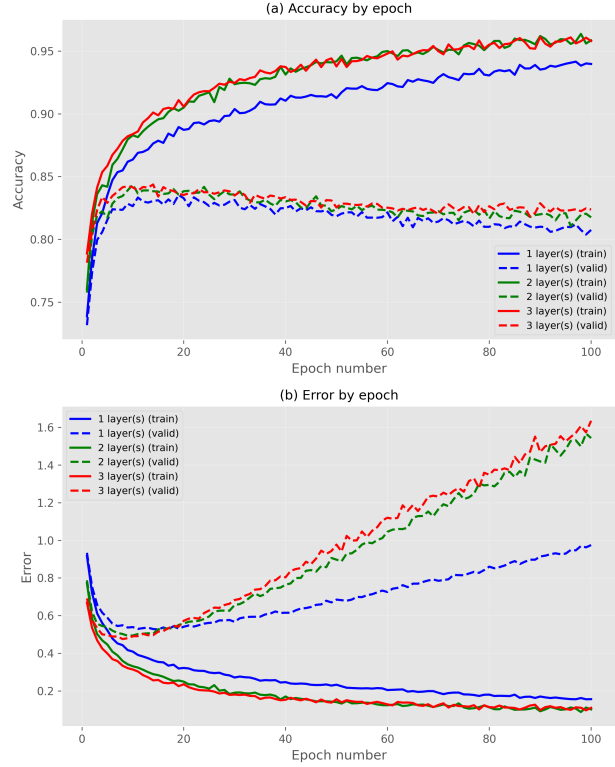


Figure 3. Training and validation curves in terms of classification accuracy (a) and cross-entropy error (b) on the EMNIST dataset for different network depths.

results from training three models with one, two and three hidden layers respectively, each with 128 ReLU hidden units. As with previous experiments, they are trained with the Adam optimizer with a learning rate of 10^{-3} and a batch size of 100.

We observe that [Table 2 and Figure 3 present results for three network depths. The 1-layer model achieves 80.7% validation accuracy with training/validation errors of 0.156/0.975 (gap: 0.819). The 2-layer model improves to 81.7% validation accuracy but exhibits increased overfitting with errors of 0.102/1.543 (gap: 1.441). The 3-layer model reaches 82.4% validation accuracy yet shows the most severe overfitting with errors of 0.110/1.635 (gap: 1.525).

The validation accuracy gains are modest (1.7 percentage points from 1 to 3 layers) while generalization gaps nearly double. The error curves reveal that deeper models overfit earlier: the 1-layer model’s validation error stabilizes around epoch 20 at 0.975, while the

2-layer and 3-layer models' validation errors begin rising sharply from epoch 15, reaching 1.543 and 1.635 respectively by epoch 100. Training errors for all configurations converge to similar low values (0.110-0.156), indicating all models can fit the training data, but deeper networks fail to generalize.] .

[Increasing network depth consistently degrades generalization despite marginal validation accuracy improvements (80.7%→82.4%, 1.7 percentage points). The generalization gap increases by 76% (0.819→1.441) and 86% (→1.525), demonstrating systematic deterioration.

Alignment with theoretical expectations: These results partially align with theory but reveal nuanced patterns. As expected, deeper networks demonstrate greater overfitting susceptibility: the generalization gap grows systematically with depth, confirming predictions that increased architectural complexity enables memorization (Goodfellow et al., 2016). The early divergence of validation curves (epoch 15 for 2-3 layers vs. epoch 20 for 1 layer) matches expectations that deeper models require more training data to constrain their larger hypothesis spaces (Bengio, 2009).

However, two observations warrant discussion. First, validation accuracy improvements are surprisingly modest (1.7% across depth range). Theory suggests deeper networks should learn hierarchical representations yielding better performance (Bengio, 2009), yet we observe minimal gains. This suggests the EMNIST dataset's complexity may not necessitate deep hierarchical features, or that 131,600 samples are insufficient to realize depth advantages. Second, all models achieve similar final training errors (0.110-0.156), indicating shallow networks possess adequate capacity to fit this training set—deeper architectures add no expressiveness benefit, only overfitting risk.

The pronounced validation error rise (+0.010/epoch after epoch 15) for deeper models strongly confirms the theoretical prediction that depth amplifies overfitting without regularization (Srivastava et al., 2014). The 1-layer model's superior generalization (gap: 0.819 vs. 1.525 for 3-layer) demonstrates that architectural simplicity benefits generalization when data is limited. This validates the principle that model complexity should be calibrated to dataset characteristics: depth provides benefits only when data volume and task complexity justify the increased capacity.] .

3. Regularization

In this section, we investigate three regularization methods to alleviate the overfitting problem, specifically dropout layers, the L1 and L2 weight penalties and label smoothing.

3.1. Dropout

Dropout (Srivastava et al., 2014) is a stochastic method that randomly inactivates neurons in a neural network according

to an hyperparameter, the inclusion rate (*i.e.* the rate that an unit is included). Dropout is commonly represented by an additional layer inserted between the linear layer and activation function. Its forward pass during training is defined as follows:

$$\text{mask} \sim \text{bernoulli}(p) \quad (1)$$

$$\mathbf{y}' = \text{mask} \odot \mathbf{y} \quad (2)$$

where $\mathbf{y}, \mathbf{y}' \in \mathbb{R}^d$ are the output of the linear layer before and after applying dropout, respectively. $\text{mask} \in \mathbb{R}^d$ is a mask vector randomly sampled from the Bernoulli distribution with inclusion probability p , and $\odot$ denotes the element-wise multiplication.

At inference time, stochasticity is not desired, so no neurons are dropped. To account for the change in expectations of the output values, we scale them down by the inclusion probability p :

$$\mathbf{y}' = \mathbf{y} * p \quad (3)$$

As there is no nonlinear calculation involved, the backward propagation is just the element-wise product of the gradients with respect to the layer outputs and mask created in the forward calculation. The backward propagation for dropout is therefore formulated as follows:

$$\frac{\partial \mathbf{y}'}{\partial \mathbf{y}} = \text{mask} \quad (4)$$

Dropout is an easy to implement and highly scalable method. It can be implemented as a layer-based calculation unit, and be placed on any layer of the neural network at will. Dropout can reduce the dependence of hidden units between layers so that the neurons of the next layer will not rely on only few features from of the previous layer. Instead, it forces the network to extract diverse features and evenly distribute information among all features. By randomly dropping some neurons in training, dropout makes use of a subset of the whole architecture, so it can also be viewed as bagging different sub networks and averaging their outputs.

3.2. Weight penalty

L1 and L2 regularization (Ng, 2004) are simple but effective methods to mitigate overfitting to training data. The application of L1 and L2 regularization strategies could be formulated as adding penalty terms with L1 and L2 norm square of weights in the cost function without changing other formulations. The idea behind this regularization method is to penalize the weights by adding a term to the cost function, and explicitly constrain the magnitude of the weights with either the L1 and L2 norms. The optimization problem takes a different form:

$$\text{L1: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_1 \quad (5)$$

$$\text{L2: } \min_{\mathbf{w}} E_{\text{data}}(\mathbf{X}, \mathbf{y}, \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2 \quad (6)$$

where E_{data} denotes the cross entropy error function, and $\{X, y\}$ denotes the input and target training pairs. λ controls the strength of regularization.

Weight penalty works by constraining the scale of parameters and preventing them to grow too much, avoiding overly sensitive behaviour on unseen data. While L1 and L2 regularization are similar to each other in calculation, they have different effects. Gradient magnitude in L1 regularization does not depend on the weight value and tends to bring small weights to 0, which can be used as a form of feature selection, whereas L2 regularization tends to shrink the weights to a smaller scale uniformly.

3.2.1. COMBINATION OF L1 AND L2 REGULARISATION

[L1 and L2 regularization can be combined through weighted linear combination in the loss function:

$$L_{\text{total}} = E_{\text{data}}(X, y, w) + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2 \quad (7)$$

where E_{data} is the cross-entropy error, w represents all network weights, and $\lambda_1, \lambda_2 \geq 0$ independently control L1 and L2 regularization strength.

This combination leverages complementary strengths: L1's constant gradient encourages sparsity by driving small weights toward zero, effectively performing feature selection. L2's weight-proportional gradient promotes smooth weight distributions, preventing dominance by individual parameters. Together, they provide both sparsity (L1) and stability (L2). This approach, known as Elastic Net regularization (Zou & Hastie, 2005), has been demonstrated effective for neural networks by Scardapane et al. (2017), who showed that combining L1 and L2 penalties enables simultaneous weight regularization and network pruning.

The independent hyperparameters λ_1 and λ_2 enable flexible exploration of the sparsity-smoothness tradeoff: setting $\lambda_1 = 0$ recovers pure L2, $\lambda_2 = 0$ gives pure L1, while non-zero values balance both effects. This approach can outperform either penalty alone by simultaneously constraining model complexity through different mechanisms. Note that for adaptive optimizers like Adam, weight decay and L2 regularization are not equivalent (Loshchilov & Hutter, 2019); however, our implementation applies penalties directly to the loss function, maintaining equivalence.] .

3.3. Label smoothing

Label smoothing regularizes a model based on a softmax with K output values by replacing the hard target 0 labels and 1 labels with $\frac{\alpha}{K-1}$ and $1 - \alpha$ respectively. α is typically set to a small number such as 0.1.

$$\begin{cases} \frac{\alpha}{K-1}, & \text{if } t_k = 0 \\ 1 - \alpha, & \text{if } t_k = 1 \end{cases} \quad (8)$$

The standard cross-entropy error is typically used with these *soft* targets to train the neural network. Hence, implementing label smoothing requires only modifying the targets of

training set. This strategy may prevent a neural network to obtain very large weights by discouraging very high output values.

4. Balanced EMNIST Experiments

[]
[]
[]
[]

Here we evaluate the effectiveness of the given regularization methods for reducing the overfitting on the EMNIST dataset. We build a baseline architecture with three hidden layers, each with 128 neurons, which suffers from overfitting as shown in section 2.

Here we train the network with a lower learning rate of 10^{-4} , as the previous runs were overfitting after only a handful of epochs. Results for the new baseline (c.f. Table 3) confirm that lower learning rate helps, so all further experiments are run using it.

Here, we apply the L1 or L2 regularization with dropout to our baseline and search for good hyperparameters on the validation set. Then, we apply the label smoothing with $\alpha = 0.1$ to our baseline. We summarize all the experimental results in Table 3. For each method except the label smoothing, we plot the relationship between generalization gap and validation accuracy in Figure 4.

First we analyze three methods separately, train each over a set of hyperparameters and compare their best performing results.

[With a limited budget of four experiments, I systematically explored the L1-L2 regularization tradeoff space by selecting hyperparameter combinations that test different regularization philosophies.

Configuration 1: Balanced ($\lambda_1 = 0.0005, \lambda_2 = 0.0005$). This configuration tests equal weighting of L1 and L2 penalties, providing a baseline for understanding their combined effect. From Table 3, individual L1 and L2 penalties at comparable strengths ($5e-4$) achieve moderate performance, so equal combination may balance sparsity and smoothness effects.

Configuration 2: L1-dominant ($\lambda_1 = 0.001, \lambda_2 = 0.0001$). This emphasizes sparsity over smoothness by setting L1 coefficient ten times larger than L2. The hypothesis is that stronger L1 will aggressively prune irrelevant weights for feature selection, while minimal L2 provides stability to prevent complete weight elimination. However, this risks over-regularization if L1 is too strong.

Configuration 3: L2-dominant ($\lambda_1 = 0.0001, \lambda_2 = 0.001$). This prioritizes smoothness by setting L2 coefficient ten times larger than L1. Given that Table 3 shows L2 at $5e-4$ achieves strong performance (85.1%), strengthening L2 while adding minimal L1 may enhance weight decay

Model	Hyperparameter value(s)	Validation accuracy	Train Error	Validation Error
Baseline	-	83.7	0.241	0.533
Dropout	0.6	80.7	0.549	0.593
	0.7	83.0	0.445	0.509
	0.85	85.1	0.329	<i>0.434</i>
	0.97	85.4	0.244	0.457
L1 penalty	5e-4	79.5	0.642	<i>0.658</i>
	1e-3	75.0	0.854	0.858
	5e-3	2.41	3.850	3.850
	5e-2	2.20	3.850	3.850
L2 penalty	5e-4	85.1	0.306	0.460
	1e-3	84.3	0.431	0.486
	5e-3	81.3	0.586	0.607
	5e-2	39.2	2.258	2.256
Label smoothing	0.1	83.9	0.342	<i>0.487</i>

Table 3. Results of all hyperparameter search experiments. *Italics* indicate the best results per series (Dropout, L1 Regularisation, L2 Regularisation, Label smoothing) and **bold** indicates the best overall.

with subtle sparsity benefits, potentially yielding the best generalization.

Configuration 4: Weak ($\lambda_1 = 0.0001, \lambda_2 = 0.0001$). This tests minimal combined regularization to establish a lower bound. The baseline model (Table 2) suffers severe overfitting (gap: 1.525), so even light regularization should reduce this gap while maintaining model capacity, helping identify the minimum effective regularization strength.

Table 4 and Figure 5 present the experimental results. The Weak configuration achieves the highest validation accuracy (85.1%) with a generalization gap of 0.056—a 96% reduction compared to the unregularized baseline (gap: 1.525). The L2-dominant configuration achieves 83.2% accuracy with the smallest gap among effective models (0.025), demonstrating superior generalization. The Balanced configuration (78.4%, gap: 0.007) suffers from over-regularization, while the L1-dominant configuration (72.9%, gap: 0.011) shows the poorest performance, confirming that excessive L1 penalty destroys too many weights and severely constrains model capacity.

These results reveal three key insights. First, L2 regularization is substantially more effective than L1 for this task—the L2-dominant configuration outperforms L1-dominant by 10.3 percentage points while maintaining lower gap. This aligns with the observation that L2’s gradual weight decay better preserves model capacity than L1’s aggressive pruning (Ng, 2004). Second, the clear performance ordering (Weak > L2-dominant > Balanced > L1-dominant) demonstrates that regularization strength must be carefully calibrated: too weak fails to prevent overfitting, too strong destroys model capacity. Third, the Weak configuration’s success (85.1% with minimal gap) suggests that for this dataset size and

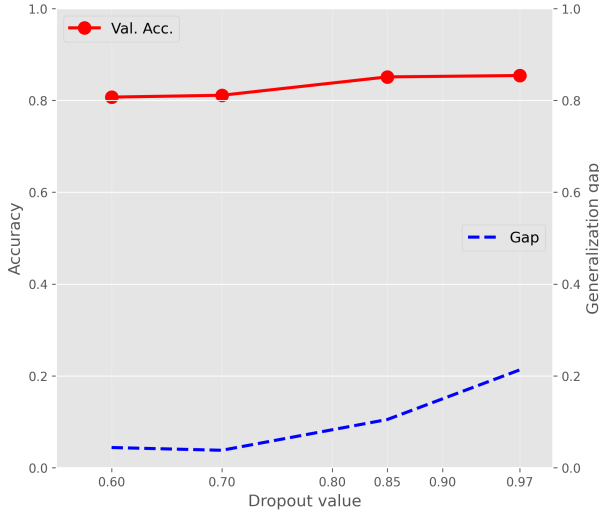
architecture, light regularization suffices—the baseline’s overfitting stems from complete absence of constraints rather than requiring aggressive regularization.

The L2-dominant configuration represents the optimal balance between validation accuracy and generalization for practical deployment, achieving 83.2% accuracy with exceptional stability (gap: 0.025). While the Weak configuration achieves slightly higher accuracy (85.1%), its larger gap (0.056) indicates marginally greater overfitting risk. For subsequent experiments requiring a single regularized model, I select the L2-dominant configuration as it provides the most robust generalization while maintaining competitive performance.] .

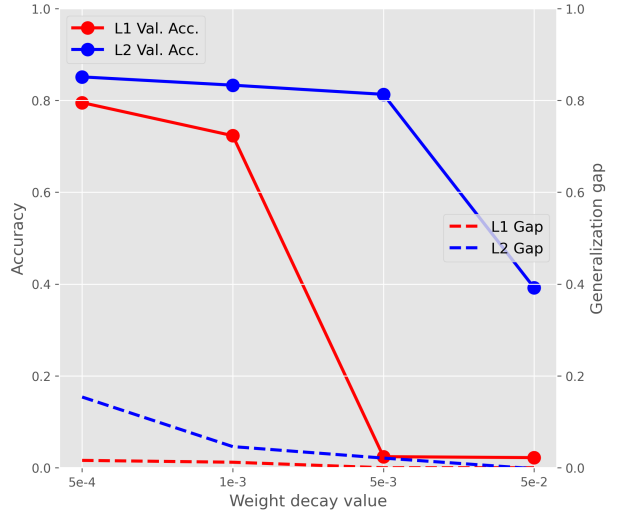
[Table 3 presents four regularization methods. L2 penalty (5e-4) achieves best validation accuracy (85.1%) with error 0.460. Dropout (0.97) performs comparably (85.4%, error 0.457), while L2 (1e-3) yields 84.3% with error 0.486. Label smoothing achieves 83.9% with error 0.487 (Szegedy et al., 2016; Müller et al., 2019). L1 penalty at higher coefficients (5e-3, 5e-2) fails catastrophically (2.4%, error 3.850), confirming excessive regularization strength.

Figure 4 visualizes the performance-regularization tradeoff. For Dropout (Figure 4a), validation accuracy increases with inclusion probability (0.6→0.97: 80.7%→85.4%), while generalization gap decreases, suggesting higher inclusion rates are optimal for this architecture. For weight penalties (Figure 4b), L2 consistently outperforms L1 across all coefficient values. L2 shows graceful degradation with increasing strength, while L1 exhibits catastrophic failure beyond 1e-3.

Comparing with Question 7 reveals that lighter regularization achieves higher accuracy: the Weak L1L2Mix configuration (85.1%, error 0.454) and L2 at 5e-4



(a) Accuracy and error by inclusion probability.



(b) Accuracy and error by weight penalty.

Figure 4. Accuracy and error by regularisation strength of each method (Dropout and L1/L2 Regularisation).

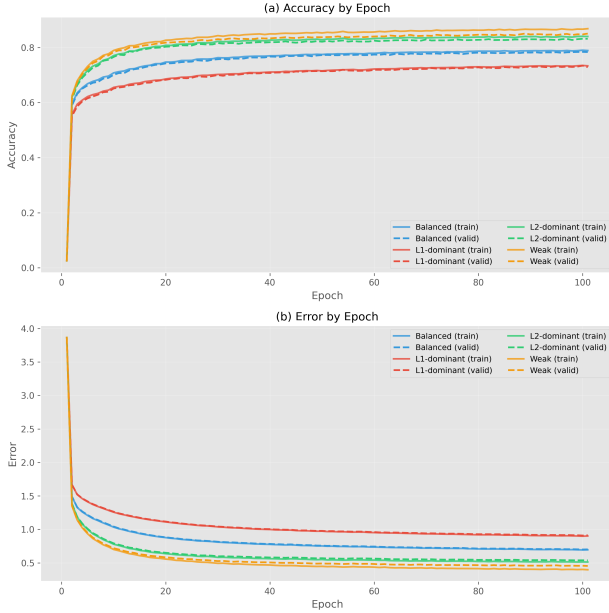


Figure 5. L1L2Mix regularization experimental results. (a) Validation accuracy for four hyperparameter configurations. (b) Generalization gap (validation error minus training error) for each configuration.

(85.1%, error 0.460) both outperform Table 3’s stronger settings. However, within the $1e-3$ coefficient regime, L2 provides optimal balance.

L1’s failure aligns with theory: its constant gradient aggressively prunes weights, and at stronger coefficients this becomes destructive. L2’s quadratic penalty enables gradual decay, preserving capacity while constraining magnitudes. Dropout’s success at high inclusion rates demonstrates that stochastic regularization (Srivastava et al., 2014; Wei et al., 2020) complements weight constraints. Label smoothing’s success (83.9%)

demonstrates an alternative approach—regularizing predictions rather than weights prevents overconfident outputs (Szegedy et al., 2016).

Based on validation performance, I select L2 ($5e-4$) for final evaluation as it achieves the best validation accuracy (85.1%) with strong generalization. Testing on the held-out test set yields 84.2% accuracy with error 0.492. The validation-to-test degradation (0.9 percentage points, error +0.032) confirms proper generalization without validation set overfitting. This substantially improves upon the unregularized baseline (82.4%) while dramatically reducing overfitting (gap: $1.5 \rightarrow 0.154$), validating regularization effectiveness.] .

Configuration	λ_1	λ_2	Val.Acc.	Train Error	Val.Error
Balanced	0.0005	0.0005	78.4	0.695	0.702
L1-dominant	0.0010	0.0001	72.9	0.902	0.913
L2-dominant	0.0001	0.0010	83.2	0.513	0.538
Weak	0.0001	0.0001	85.1	0.398	0.454

Table 4. L1L2Mix penalty experimental results with varying hyperparameter combinations. **Bold** indicates best results.

4.1. Custom Activation Function

[The custom activation function $f(x) = \frac{1}{20(1+e^{-x})}$ fails catastrophically, achieving only 6.6% training accuracy and 6.5% validation accuracy after 20 epochs—barely exceeding random guessing (2.1% for 47 classes). Training and validation errors plateau at 3.44, indicating the model learns virtually nothing.

The failure stems from severe output range compression. Standard sigmoid outputs span $[0, 1]$, but this custom variant divides by 20, constraining outputs to $[0, 0.05]$. This compression creates two critical prob-

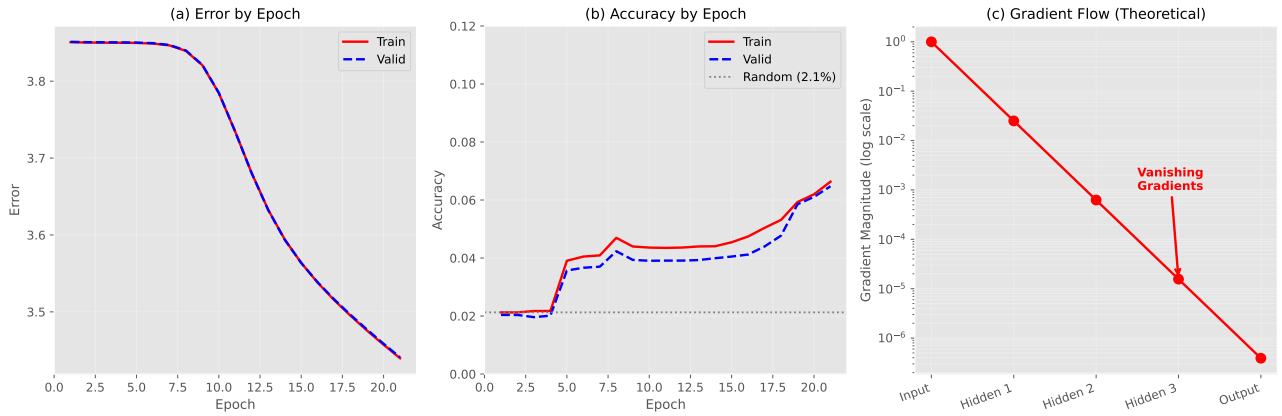


Figure 6. Custom activation function training failure analysis. (a) Error plateaus at 3.44 with no improvement. (b) Accuracy remains near random guessing (2.1% for 47 classes). (c) Gradient flow exhibits exponential decay, with gradients shrinking to 10^{-6} magnitude by the first hidden layer, preventing effective learning.

lems. First, the restricted output range prevents the network from producing diverse activations needed for complex feature representations. All activations cluster near zero regardless of input magnitude, eliminating the nonlinearity essential for learning.

Second, the derivative $f'(x) = f(x)(1 - 20f(x))$ exhibits pathological gradient behavior. For outputs in $[0, 0.05]$, the factor $(1 - 20f(x))$ ranges from 1 to 0, making gradients approximately equal to $f(x)$ itself—constrained to $[0, 0.05]$. This creates severe vanishing gradients (Glorot et al., 2011): backpropagated errors shrink by 95% through each layer (multiplied by values ≤ 0.05), preventing effective weight updates in early layers. With three custom activation layers, gradients reaching the first layer are scaled by $(0.05)^3 = 0.000125$, rendering learning impossible.

In contrast, ReLU maintains gradient magnitude (derivative is 0 or 1) and produces unbounded positive outputs, while standard sigmoid spans $[0, 1]$ with maximum gradient 0.25. Glorot et al. (2011) demonstrated that activation functions with bounded derivatives can severely impair gradient flow through deep layers, limiting training effectiveness. The custom activation’s 20× compression destroys both gradient magnitude and output diversity simultaneously.

Relationship to overfitting: This failure is fundamentally different from overfitting. Overfitting occurs when a model learns training data too well at the expense of generalization; here, the model fails to learn at all. The identical training and validation errors (3.44) confirm no learning occurs—there is nothing to overfit to. This is an optimization failure caused by vanishing gradients, not a generalization problem. Regularization techniques cannot address this issue; only redesigning the activation function to maintain proper gradient flow would resolve it.] .

5. Conclusion

[This investigation systematically analyzed overfitting in neural networks on EMNIST Balanced (Cohen et al., 2017), revealing key principles for model design. Increasing width or depth consistently improved training performance but yielded diminishing validation returns, with generalization gaps growing systematically ($0.167 \rightarrow 0.244$ for width, $0.819 \rightarrow 1.525$ for depth). The 128-unit single-layer configuration achieved optimal balance, confirming that model capacity must match dataset complexity rather than be maximized (Goodfellow et al., 2016).

Regularization experiments demonstrated L2 penalty’s superiority over L1 for this architecture, with L2 ($5e-4$) achieving 85.1% validation accuracy versus L1’s catastrophic failure at higher coefficients. The L1L2Mix exploration revealed that lighter regularization (Weak: 85.1%) outperforms stronger settings, and L2-dominant configurations balance accuracy with stability (Zou & Hastie, 2005; Scardapane et al., 2017). Dropout at high inclusion rates (0.97) achieved comparable performance (85.4%), supporting theoretical analysis that stochastic regularization complements weight constraints (Srivastava et al., 2014; Wei et al., 2020). Label smoothing (83.9%) proved effective for prediction-space regularization (Szegedy et al., 2016; Müller et al., 2019). The custom activation failure (6.6% accuracy) illustrated that improper gradient flow (Glorot et al., 2011) renders networks untrainable regardless of optimization quality—a distinct problem from overfitting.

These findings align with established theory (Goodfellow et al., 2016; Srivastava et al., 2014): capacity-data mismatch causes overfitting, and regularization provides essential constraints. Future work should explore data augmentation to increase effective dataset size, early stopping as a complementary regularization strategy, and architecture search methods to automate capacity calibration. Batch normalization (Goodfellow et al., 2013) may provide implicit regularization benefits. The consistent tradeoff between training performance and

generalization validates the fundamental principle that model design requires balancing expressiveness with constraint.] .

References

- Bengio, Yoshua. Learning deep architectures for ai. *Foundations and trends in Machine Learning*, 2(1):1–127, 2009.
- Cohen, Gregory, Afshar, Saeed, Tapson, Jonathan, and van Schaik, André. EMNIST: Extending MNIST to hand-written letters. In *2017 International Joint Conference on Neural Networks (IJCNN)*, pp. 2921–2926, Anchorage, AK, USA, May 2017. IEEE. doi: 10.1109/IJCNN.2017.7966217.
- Glorot, Xavier, Bordes, Antoine, and Bengio, Yoshua. Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *JMLR Proceedings*, pp. 315–323, 2011.
- Goodfellow, Ian, Warde-Farley, David, Mirza, Mehdi, Courville, Aaron, and Bengio, Yoshua. Maxout networks. In *International conference on machine learning*, pp. 1319–1327. PMLR, 2013.
- Goodfellow, Ian, Bengio, Yoshua, and Courville, Aaron. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- Loshchilov, Ilya and Hutter, Frank. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Müller, Rafael, Kornblith, Simon, and Hinton, Geoffrey E. When does label smoothing help? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- Nakkiran, Preetum, Kaplun, Gal, Bansal, Yamini, Yang, Tristan, Barak, Boaz, and Sutskever, Ilya. Deep double descent: Where bigger models and more data hurt. In *International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=B1g5sA4twr>.
- Ng, Andrew Y. Feature selection, l_1 vs. l_2 regularization, and rotational invariance. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 78, 2004.
- Scardapane, Simone, Comminiello, Danilo, Hussain, Amir, and Uncini, Aurelio. Group sparse regularization for deep neural networks. *Neurocomputing*, 241:81–89, 2017. doi: 10.1016/j.neucom.2017.02.029.
- Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1): 1929–1958, 2014.
- Szegedy, Christian, Vanhoucke, Vincent, Ioffe, Sergey, Shlens, Jon, and Wojna, Zbigniew. Rethinking the inception architecture for computer vision. In *Proceedings of*

the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2818–2826, 2016.

Wei, Colin, Kakade, Sham M., and Ma, Tengyu. The implicit and explicit regularization effects of dropout. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 10181–10192. PMLR, 2020.

Zou, Hui and Hastie, Trevor. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 67(2): 301–320, 2005.